

Versioning Paradigms — Multi-Level Identity & Versioning Beyond Git

Research synthesis, 2026-06-11. Question we are answering: what is the actual identity of code (and of any structured material) once you stop treating a file as the atom — and how do you version it at multiple granularities *at once* (a function, a grouping of functions, a graph at graph / node / relation-type / node-data level), where groupings are themselves versioned objects with histories partly independent of their members?

How to read this doc. Each source gets three movements: ① **What the paper actually worked through** (the substance — mechanisms, findings, the argument, with the authors' own words) · ② **How it maps to our question** · ③ **What it means for the Softland kernel.** The cross-cutting synthesis is at the end — *earned* by the per-source detail, not asserted ahead of it.

Calibration. 25 sources fetched, 119 claims extracted by background agents; the adversarial verify pass was cut for token budget, so these are *extracted, not independently verified* claims — weigh by source quality (most are primary papers/docs), and treat single-source claims with care. Raw material:

`~/.claude/.../workflows/scripts/research-salvage.json`. Named-but-not-fetched (a second pass would add them): Stellation, ENVY/Epicea, Software Heritage SWHIDs, GumTree/AST-diff, RefactoringMiner, Automerge/Yjs history models, Okasaki, Xanadu primary sources, codeq itself (known from its seed blog post).

Angle 1 — Program-unit identity: what *is* a function, across time?

Unison — identity is the content, names are metadata

unison-lang.org/docs/the-big-idea — primary, design stable since ~2019

① **What it worked through.** Unison takes the most extreme position available: a definition has no name-identity and no location-identity at all. "Each Unison definition is identified by a hash of its syntax tree" — a 512-bit SHA3 computed *after* normalizing away the incidental: "all named arguments are replaced by positionally-numbered variable references, and all dependencies ... are replaced by their hashes." So two functions that differ only in variable names or formatting hash to the same address; the hash captures *meaning*, not text. Names then live in a completely separate layer: "names are just separately stored metadata that don't affect the function's hash ... we can change what address a name points to, but the contents of each address are forever unchanging." The whole codebase becomes "a proper database of code" — hash-indexed, immutable, so a parse/typecheck cache "is never invalidated." And versioning composes by transitive pinning: "the hash of `increment` uniquely identifies its exact implementation and pins down all its dependencies," which is why "dependency conflicts and the diamond dependency problem are just not a thing" — every version of every function coexists under a distinct hash.

② **How it maps.** This is the purest answer to your opening question ("what is the identity of a code file") — and it says: *not the file, not the path, not the name — the normalized AST*. Rename becomes a **declared metadata edit** by construction (you rebind a name → hash pointer), never an inference. It is the markdown ingestor's content-addressed identity, but lifted from raw bytes to *semantic* form, so trivial edits don't churn identity.

③ **What it means.** Unison is the destination pole of the two worlds we named — "the world IS the codebase." It proves the function-grain identity you sketched is real and buildable. But it also shows the cost of going all the way: you must abandon files as truth, abandon the existing toolchain, and mint a whole new naming layer — which is exactly the bridge the kernel should keep *possible* but not *cross* in slice 1. The transferable idea now: **identity should be content-derived at the semantic unit, with a thin mutable name layer on top** — and Softland's revisions already are the value layer, its containers already the name layer.

Molhado — minted node identity, versioning opt-in per attribute

Nguyen, SCM-12 ch.9 (2005) — primary; the single closest system to the kernel

① **What it worked through.** Molhado (Tien N. Nguyen) is built on a data model called Fluid IR that separates the Hickey triple *mechanically*: "a node is the basic unit of identity ... a slot is a location that can store a value ... attributes map nodes to slots." Identity (node), value (slot contents), place (which attribute table). The move that matters: **versioning is opt-in per attribute** — "slots in an attribute table can belong to three types: constant, simple, or versioned" — so *which aspects of a thing have history* is a modeling decision, and turning on versioning just adds a third dimension (the version axis) to the node×attribute table. On top of this, Molhado uses a **product-versioning** model: "instead of focusing on individual components, Molhado versions a software project as a whole ... all system objects ... are versioned in a uniform, global version space," where a version is "a point in tree-structured discrete time for the whole project." And one mechanism covers every granularity at once: "Molhado can track changes ... not only of a hypertext network but also of a single node or link" — because *links are themselves first-class nodes*, so relation history "falls out for free."

The chapter also names the field's taxonomy and its hazard: three paradigms — **composition** (version atoms, assemble by rule), **total versioning** (everything including composites has its own version space), **product versioning** (one global space) — and flags that total versioning "suffers from the version proliferation problem": one leaf change forces a new version of every ancestor composite. And it flags *its own* unsolved problem: because everything lives in one global version, "there is no way to say 'make a link to document D at version vk'" — **cross-version references are inexpressible**.

② **How it maps.** This is your function-as-atom sketch, already built and measured. "Group a whole project, version a single node or a single link, with one mechanism" is precisely the multi-level versioning you reached for. Per-attribute opt-in is the answer to "what aspects of a thing get history" — edit history, semantic history, relational history can each be a versioned slot or not.

③ **What it means.** Two gifts and one loud warning. Gift one: **node-identity that is not the name** — the thing the kernel calls a container, with the name demoted to a versioned attribute. Gift two: **per-attribute versioning** maps cleanly onto Rama PStates — you choose which projections carry history. The warning is load-bearing for Softland specifically: a *single global version space breaks anchors and trails*, because they fundamentally need "link to D at version vk." So adopt Molhado's identity model, but **not** its product-versioning — the kernel needs per-thing version threads *and* cross-version reference, which points at the composition model (see Katz/ORION).

MolhadoRef — declared refactorings beat inferred diffs

Nguyen et al., ASE (2006) — primary; the precedent for our "no inferred truth" rule

① **What it worked through.** MolhadoRef puts the Molhado identity model to work against the rename problem and reaches a sharp result: "Molhado ... creates unique, persistent IDs for each entity, and treats the name as an attribute that can change. Thus it can track the history of an entity in spite of it being refactored." The architecture is a **hybrid change representation**: ordinary edits are captured as state deltas (via the Eclipse compare engine), but *refactorings are recorded as first-class operations* — "it uses the operation-based SCM approach to represent and record refactoring operations as first-class entities in the repository," **declared by the refactoring engine, not inferred by diffing**, and replayed on update/merge. The payoff is concrete: on three 2006 revisions of an Eclipse component, "MolhadoRef correctly retrieves the history of classes and methods renamed or moved ... while CVS loses their history," at a storage cost of "2.21, 2.69, and 2.80 times ... the initial source." It also draws a deliberate granularity floor — "for efficiency reasons we stop at the method and field declaration level" — and names semantic merge (merging refactoring ops with ordinary edits) as unsolved future work.

② **How it maps.** This is the empirical proof of the kernel's own deepest rule: *inferred structure must not be accepted as truth at ingest*. Git's rename detection is the inference path (a similarity heuristic); MolhadoRef is the declaration path (the tool that did the rename records it). Same problem, opposite epistemics — and the declaration path keeps history the inference path loses.

③ **What it means.** This reframes the trail-to-code join from "resolve dead path strings" into something much larger, and it's the most important single connection in this whole research. **Softland already has the declared-operation channel that MolhadoRef had to own an IDE to get.** Agent trails record each Edit/Write as a structured act *with the intent narrated in language*. So the kernel's parked "rename continuity = explicit relation/claim policy later" has a natural producer: the agent that renamed *declared it*, recoverable from the trail as a candidate claim. We get MolhadoRef's advantage without owning everyone's editor — because the agents are already inside the world.

Monticello — sub-file versioning that stayed snapshot-based

Nierstrasz et al. (2013) — primary; the Smalltalk lineage, pragmatic end

① **What it worked through.** Monticello (Pharo/Squeak) rejects line-grain outright: "classes and methods, rather than lines of text, are the units of change." A package is an *intensional* container — "a package is an object — an instance of PackageInfo — that knows how to identify the classes and methods that belong to it" — and identity is separated from value: a version is "an immutable snapshot," version numbers are repo-local, and global identity rides on "a UUID and explicit named ancestors" (merges list multiple ancestors). Notably it is **state-based at sub-file grain**: "a version contains the complete set of class and method definitions," and patches are *derived* between snapshots, not stored as primary. Merge is fine-grained anyway — conflicts are resolved per method/class definition — and it does versioned-grouping-of-members: a package version "declares required package versions," new versions auto-rebind, and an empty "umbrella" package acts as "a pure versioned configuration whose own history is partly independent of its members." Its flagged limit: dependencies can't cross repos.

② **How it maps.** Direct precedent for "the function/method is the unit, the file is not." And it answers a question your sketch raised — *must sub-file versioning be delta-based?* No: Monticello stores snapshots at method grain and derives diffs. The umbrella package is exactly your "grouping of functions, versioned, with its own history."

③ **What it means.** Two calibrations. First, it de-risks the slice-1 choice: storing whole-unit snapshots and deriving diffs is a *proven* sub-file strategy, not a compromise — so the kernel's whole-file-artifact-now, forms-

later plan is sound. Second, the umbrella package is the small, concrete shape of a Softland "configuration" (a versioned grouping) — and its cross-repo limitation is a feature to *beat*, since Softland's containers are global, not repo-scoped.

Angle 2 — Version-control theory: what is versioning, formally?

Driscoll, Sarnak, Sleator & Tarjan — the formal taxonomy of "versioned"

"Making Data Structures Persistent," JCSS (1989) — primary; the bedrock

① **What it worked through.** This is the paper that defines the words. "A structure is *persistent* if it supports access to multiple versions. It is *partially persistent* if all versions can be accessed but only the newest version can be modified, and *fully persistent* if every version can be both accessed and modified." Partial persistence = a linear log (read history, extend the tip). Full persistence = a branching tree of immutable versions: "the result of the update is an entirely new version, distinct from all others ... versions form a tree." And the engineering miracle: this is nearly free. "If an ephemeral structure has nodes of constant bounded in-degree, then the structure can be made partially persistent at an amortized space cost of $O(1)$ per update step and a constant factor in ... time" — fully persistent with more work. The whole field traces to "the second author's Ph.D. thesis" (Neil Sarnak) and one prior Overmars paper.

② **How it maps.** This is the precise vocabulary for your "versioning is its own layer" intuition. Rama's arrival-time history is **partial persistence** — a log, read any past state, only the tip advances. The explicit git-style committed versioning you want is **full persistence** — branchable, every version a first-class point you can fork from. They are different formal objects, and you were right to feel the seam.

③ **What it means.** Names the two-layer model exactly: the depot (partial persistence, transaction time, append-only substrate) *underneath* a minted, branchable version graph (full persistence, semantic time). "Infinite Softlands in Rama" is, literally, *full persistence* — and DSST proves it costs $O(1)$ amortized space via structural sharing, which is also why every survivor in Angle 3 can afford to keep all of history.

Conradi & Westfechtel — the survey that already answered the whole question

ACM Computing Surveys (1998) — primary; the spine of the field

① **What it worked through.** This survey unifies thirty years of SCM and is startlingly on-point. Identity through time is defined as a **sameness criterion**: "there must be some way to decide whether two versions belong to the same item ... with the help of a unique identifier, for example an OID," while each version carries its own VID — and a version is formally "a pair (ps, vs)" of a product-space state and a version-space point (value and place, split). Crucially, the **item** — the thing versioned — is *anything*: "files and directories ... objects ... entities, relationships, and attributes in EER databases." So your four-level graph case is *inside* the definition. It names **multilevel / recursive versioning** ("a version may have versions themselves") as the solution to versioned-groups-of-versioned-members, citing the seminal systems — **Adele** (Estublier 1985: interface versions each realized by a set of body versions) and **DAMOKLES** (Dittrich

et al. 1986: fully recursive). It formalizes group $\leftrightarrow$ member binding via **AND/OR graphs** — a reference is **bound** (pinned) or **generic** (floating); a configuration is bound iff all edges are. It classifies models by selection order — *product-first* (SCCS/RCS: structure fixed, can't version structure), *version-first* (PCTE: group version pins members and can change structure), *intertwined*. And it nails change representation: "a *directed delta*, also called a change, is a sequence of (elementary) change operations ... (note the correspondence to transaction logs in databases)" — versus symmetric (state) deltas — and declares state-based vs change-based "orthogonal to extensional versus intensional."

② **How it maps.** Almost every noun in our chat is here under an older name: container = item/OID, revision = VID, "versioning is its own layer" = the version space as a distinct formal space, "grouping of functions versioned" = multilevel versioning, "pinned vs floating members" = bound vs generic. The event-log-as-primary intuition is their *directed delta* = *transaction log* equation.

③ **What it means.** We are not pioneering; we are re-deriving a known framework with better tools. The cheapest highest-value move available is to **adopt this vocabulary wholesale** in the kernel docs — it will make every future ingestor contract and every Codex gate dramatically cheaper to argue, because the terms are precise and the failure modes are already named.

Darcs patch theory — change as the primary object

Understanding Darcs/Patch Theory (Wikibooks) — secondary

① **What it worked through.** Darcs inverts git: the patch, not the snapshot, is primary. A patch is context-dependent — it "tells us not only what contents Arjan added ('beer') but where ... namely to line 3" — so a patch only means anything relative to the state it transforms. The algebra has two axioms: **invertibility** ("every patch must have an inverse" with PP^{-1} a no-op) and **commutation** — merge is defined by reordering patches while rewriting their context: "what used to be line 3 now becomes line 4." The formal merge law is $BA_1 \leftrightarrow AB_1 \text{ iff } B_1A_1^{-1} \leftrightarrow A^{-1}B$. The treatment honestly covers only non-conflicting merges and admits non-commutable pairs ("you can't commute the addition of a file with adding contents to it") — conflict semantics deferred.

② **How it maps.** This is the operation-log view of versioning at the formal level — the same shape as MolhadoRef's declared operations and as event sourcing. Identity here lives in *the sequence of changes*, not in any snapshot.

③ **What it means.** Useful as a lens, not a build: it tells us the op-log (agent trails) is a legitimate *primary* representation of change, with snapshots derivable — but its own unfinished conflict story warns that operation-based merge is hard. Keep snapshots as truth; treat the op-log as the explanatory/semantic layer over them.

Pijul — categorical merge and sub-file graph identity

pijul.org/model — primary

① **What it worked through.** Pijul gives patch theory a cleaner foundation. Merge is a **pushout** — "a canonical common successor state from which anything derivable from either branch can also be derived" — making merge a characterized operation, not a heuristic. Below the file, "a file is represented as a directed graph of byte chunks" where chunks are the versioned units and "the graph edges [are] the order in which they appear in the files" — sub-file identity with structure as explicit relations. The deep move: **conflicts are**

represented as values, by enlarging the state space — "artificially add all pushouts of all possible patches" — so a conflicted state is a storable, first-class thing, not an error. A version is "an unordered set of patches," which avoids git's artificial cherry-pick conflicts, and caching applied-patch state in the chunk graph gives $O(\log h)$ application (vs Darcs checking each patch against all prior).

② **How it maps.** Two things. The byte-chunk-graph is the same instinct as treating forms as nodes in a relation graph. And "conflict as a first-class value" is the formal twin of Softland's **preserved disagreement until synthesis** — late-bound consensus is, mathematically, refusing to collapse a pushout that doesn't exist yet.

③ **What it means.** The most directly *philosophical* payoff in the haul: your "preserved tension / plurality as a fourth axis" is not just a design value — Pijul shows it can be a **data structure**. A conflict (a disagreement between sources, a merge not yet resolvable) should be a representable state in the kernel, not an error or a silent overwrite. That's a real design principle for the synthesis-of-plurality artifact type.

Homotopy patch theory — identity = history modulo commutation

Angiuli, Morehouse, Licata, Harper (2014) — primary

① **What it worked through.** This encodes Darcs-style patch theory in homotopy type theory: "patch contexts are represented as points ... patches are represented as paths between ... contexts," and patch laws (commuting independent edits) are "2-dimensional paths between paths." Because in HoTT "functions are functors that necessarily preserve the path structure," any interpretation of a patch theory is *forced* to validate its laws — "one patch theory can have many models maintaining different metadata." Their most expressive language makes repository identity **event-sourced**: "complete histories ... uniquely identify files because the history can be replayed from the start," with histories "quotiented so that sequences of patches with the same effect ... are equal." And they are candid about limits: the merge construction "is incomplete," the required invertibility of all patches is "conceptually questionable and practically problematic," and they could not formalize Darcs' pseudocommutation — pointing to directed type theory as future work.

② **How it maps.** This is the formal statement of "the event log is primary, rendered state is a projection" — *one theory (the log + its laws), many models (materialized views)*. And "identity = complete history quotiented by commutation of independent events" is exactly the right formal picture for **semantic time as a DAG, not a line**: two independent edits commute, so they're unordered; dependent ones don't.

③ **What it means.** It dignifies the kernel's core architecture as a theorem rather than a preference: log-primary + derived projections is the *natural* model, and parallel independent events being order-free is *why* semantic time is a DAG. It also flags the real frontier (merge laws, non-invertible changes) so we don't promise automatic semantic merge we can't deliver.

Angle 3 — Versioned databases: the two-layer pattern, in production

Irmin — content-addressed values + a mutable name layer, as a library

The Morning Paper on Farinier et al. (2015) — secondary

① **What it worked through.** Irmin is git's architecture extracted as an OCaml library over arbitrary data structures. The identity/value split is explicit at the storage layer: "the key returned from a store operation is simply a hash of the stored value" (immutable content-addressed block store), while "[the tag store] keys are names created by users and values are keys from the block store ... named pointers." It enumerates exactly three ways to merge diverged structured data: "(1) using CRDTs (conflict-free) (2) a data-structure-specific merge operation (3) pushing the problem onto the developer." And merge **composes recursively across levels**: a 3-way merge "descends to the smallest subtree where divergence occurred," auto-resolves one-sided changes at the container level, and only "flattens the sub-trees into a user-provided container with a data-type-specific merge" when both sides changed. It maintains a bidirectional mapping to git's on-disk format and runs as the file system of MirageOS, at "about a factor of 5" overhead.

② **How it maps.** This is the kernel's container/revision split, validated as a shippable storage pattern — and the recursive merge is the answer to "how does group-version relate to member-version on merge": *the group merge delegates to member merge only where members actually diverged.*

③ **What it means.** Strong reassurance that the two-layer model isn't a Softland invention to be doubted — it's the convergent answer (git, Irmin, Unison, Dolt all reach it). And the three-way merge taxonomy (CRDT / typed-merge / defer-to-human) is a ready menu for when the kernel eventually merges divergent local worlds.

Dolt + Prolly trees — content-defined structure, diff $\propto$ change

DoltHub docs + blogs (2020, 2021) — primary docs / blog

① **What it worked through.** Dolt versions a SQL database at **cell granularity** — "cell-wise diffs and merges" — using a **Prolly tree** (probabilistic B-tree, from Noms): chunk boundaries are computed *from the content* — "use a seed, the size of the current chunk, the key value and a strong hash to calculate whether this ... represents a new chunk boundary" — so "identical data always produces identical ... structure," chunks are stored once by hash ("any blocks that share the same content address are only stored once"), and diff "scales with the size of the differences, not the size of the tree." It transplants git's commit/branch/merge model below the file ("the Git command line ... on table rows instead of files"). The blogs define the category — "a version-controlled database ... stores a full history of its own state, both data and schema" — credit Merkle trees as the enabling structure, and note schema-vs-data are handled by *disjoint* tooling (migrations for schema, Dolt for data).

② **How it maps.** Prolly trees are the data-structure answer to "content-derived identity that also diffs cheaply" — the same sha-dedup as codeq's forms, but with efficient structural diff for free. And cell-granularity proves fine-grained versioning is practical at production scale.

③ **What it means.** The concrete recommendation that came out of this for slice 1: **dedup artifacts by content, not per (path, version)** — store one value per distinct bytes, link occurrences. Same bytes at a new path = same value, new place, which hands us *half of rename continuity as fact* (not inference) for free. Prolly/Merkle structural sharing is how you afford to keep everything.

XTDB — bitemporality: "when it was true" vs "when we learned it"

v1-docs.xtdb.com/concepts/bitemporality — primary docs

① **What it worked through.** XTDB versions along two independent axes: **valid-time** ("an arbitrary time that can originate from an upstream system, or by default is set to transaction-time") and **transaction-time** (system ingestion). Transaction-time is strictly append-only — "you cannot write a new transaction with a transaction-time in the past" — an immutable audit trail. Valid-time is *correctable*: "writes can be made in the past of valid-time as retroactive operations ... or the future as proactive operations," without destroying the audit record. The docs explicitly ground this in DSST: "XTDB's bitemporal indexes are partially persistent due to the immutability of transaction time." Granularity is whole-document with a stable `xt/id` and full-snapshot replacement.

② **How it maps.** This is the exact formal home of the code-ingestor's **exactness problem**. "When the agent saw the bytes" is valid-time; "when Softland ingested the commit" is transaction-time. The contract's exactness flag is a one-bit projection of a two-axis bitemporal model.

③ **What it means.** Adopt the two times explicitly in the kernel: commit author-time (valid) and ingest-time (transaction) as separate fields. The exactness flag then becomes a *consequence* of the model rather than a special case, and "corrected truth layered over an immutable log" (retroactive valid-time) is precisely how a trail's later, more-accurate content should reconcile against an earlier approximate resolution. The map stays honest by construction.

TerminusDB — git-for-graphs with append-only delta layers

terminusdb.org/docs — *primary docs*

① **What it worked through.** TerminusDB is a graph database with git semantics: "every commit appends a new delta layer — an immutable record of what was added and what was removed ... TerminusDB never mutates existing data," each layer "content-addressed and immutable" (enabling lock-free reads). Diff/patch operate at **two composable granularities** — "at the document or triple level" — and it offers "branch, diff, merge, clone, and time-travel ... like Git, but for structured data," decomposing JSON documents into RDF triples at ~13 bytes/triple with periodic rollups.

② **How it maps.** A production existence-proof that git's *model* (not its file assumption) lifts onto graphs, with diff at two levels at once — the multi-granularity your graph question wants. Append-only immutable delta layers = event sourcing for graphs.

③ **What it means.** When the Roam/graph ingestor arrives, "git-for-graphs" is a built reference point: immutable content-addressed layers + document-and-triple-level diff is a viable shape on top of an event-sourced substrate — and it's close to what Rama gives natively.

Angle 4 — Graph & RDF versioning: the multi-level case, stress-tested

RDF archiving surveys — snapshot vs delta vs annotation, and where they break

SW journal SW-180309 (2019) + swj2538 (2021) — primary

① **What they worked through.** The field reduces to three storage paradigms: **independent copies** (IC, full snapshots), **change-based** (CB, stored triple deltas), **timestamp-based** (TB, each triple annotated with version validity) — plus Quilt Store's **fragment-based** and hybrids. The foundational model versions at *exactly one* granularity: "a version-annotated triple (s,p,o):[i] ... an RDF archive graph A is a set of version-annotated triples" — no separate identity thread for nodes or subgraphs. Empirically, **no paradigm dominates**: CB/TB beat snapshots on storage for low-change data but are "penalized in highly dynamic datasets," and CB query time degrades as delta chains lengthen. And fine granularity *breaks the systems outright*: versioning DBpedia at instant grain (21,046 versions) made TB implementations "fail to load," and an R43ples query "had to be stopped after a 6-hour timeout" at 1,299 versions. The 2021 survey adds the multi-level result: a triple occurrence needs a 5-tuple $\langle s, p, o, p, \zeta \rangle$ — p a *local* revision of the named graph, ζ a *global* dataset revision — and proves systems with only graph-local revisions "cannot track addition/deletion of named graphs," i.e. **member-level versioning alone cannot reconstruct group-level history**. It also argues change semantics must lift from triples to entities ("high-level changes"). The literature flatly states the area is immature: "there is neither a standard language to query RDF archives, nor an agreed way for ... querying temporal graphs."

② **How it maps.** This is your four-level graph question, attempted by a whole research community — and the honest report is: the naive single-grain model (version the triple, done) is *insufficient*, and you provably need at least two linked tracks (member + group) to answer group-level history.

③ **What it means.** Two hard constraints land on the kernel. First, **don't version at event/save grain** — the version-count wall (systems dying at 21k versions, 6-hour timeouts) is a real engineering ceiling, which independently vindicates commit-grain over save-grain. Second, **you must carry both a local and a global revision track from the start** — retrofitting the group-level track later is provably lossy. That's a direct argument for getting the configuration/group-version noun into the kernel before the graph ingestor, not after.

Wikidata history — braided per-entity and global chains, in production

Pellissier-Tanon & Suchanek, ESWC (2019) — *primary*

① **What it worked through.** This system stores Wikidata's full history queryable with plain SPARQL, holding **both** representations over one substrate: "we index not just the diffs ... but also the global state ... one named graph per revision." Identity threads at **two composable levels**: each revision's metadata records "the previous revision of the same entity [and] the previous revision in Wikidata" — per-entity chains *braided inside* the dataset-level chain. A fact's temporal extent is stored as **validity ranges** rather than copies: each triple lives in three permuted indexes "as value a set of revision ranges ... [start, end]" — avoiding the per-version blowup. This is efficient because history is nearly monotone: "out of 4931M triples that have existed ... only 475M have been removed ... the largest revision contains 90% of all triples that ever existed." Edits are fine-grained by nature — "each edit affects a single entity and usually a single statement."

② **How it maps.** The production answer to the 2021 survey's theorem: it actually *implements* the two linked tracks (entity-level + global-level), and it shows snapshot and delta are **both** derivable from one substrate — "snapshot vs delta" is an index decision, not an ontology decision.

③ **What it means.** A concrete, working template for Softland's eventual graph/Roam ingestor: per-node version threads braided inside a global version thread, with fact lifetimes as intervals over an append-only log. And the monotonicity datum is encouraging — knowledge that mostly accretes (which trails and discourse graphs do) is exactly the regime where interval-annotated event logs stay cheap.

OSTRICH — one concrete hybrid, and its honest scaling wall

Journal of Web Semantics (2018) — primary

① **What it worked through.** OSTRICH is a single-history hybrid of IC+CB+TB: "intermittent fully materialized snapshots are followed by delta chains ... six tree-based, dictionary-encoded, timestamped indexes," with each delta aggregated against the nearest snapshot (not chained to the prior delta) to buy constant-time random version access. It defines three reusable query atoms: **version materialization** VM (query one version), **delta materialization** DM (changeset between two versions), **version query** VQ (annotate results with the versions they hold in). It is single-linear-history only (no branch/merge), and flags its own scaling failure: aggregated changesets "can only grow in size," so ingesting a *later, smaller* revision took longer than an earlier larger one (rev 9 ~22h vs rev 5 ~14h) — "OSTRICH should not be used when the number of versions is very large."

② **How it maps.** Less about identity, more about the **query model** any versioned store needs. VM/DM/VQ is precisely the kernel's "code-at-time" (VM), "what changed between these commits" (DM), and "in which revisions did this hold" (VQ).

③ **What it means.** Steal the query vocabulary — VM/DM/VQ is a clean, source-agnostic API for the kernel's read models, and trail-to-code resolution is a VM with an exactness flag. And the scaling caveat reinforces, a third time, that aggregated-delta-against-snapshot has a real cost curve — design the read models so snapshots can be re-materialized, don't let delta chains grow unbounded.

Angle 5 — Composite/assembly versioning: the mature, boring, solved part

Katz — configurations as first-class versioned objects

ACM Computing Surveys (1990), via the 1988 Berkeley TR — primary; highest-yield for groupings

① **What it worked through.** Katz's framework separates identity/value/grouping cleanly: a **design object** is "a generic structural object with no file of its own" (the identity thread), a **version** is "a semantically meaningful snapshot ... a descendent of some existing versions," and a **configuration** is a *first-class binding* between a composite version and specific component versions. It makes pinned-vs-floating a deliberate axis: **static** configurations pin ("a composite object version carries with it the version numbers of its components") while **dynamic** ones defer binding to traversal-time, via "layers and contexts" borrowed from Goldstein & Bobrow's PIE system at Xerox PARC (1981) — and Katz flags the weakness that dynamic contexts can compose incompatible component versions. He identifies the cross-level hazard precisely: "even though A's contents have not changed, it now points to a new component, and is thus a new version" — **change propagation** ripples upward, is path-ambiguous through DAGs, and his fix is *operational*: "group check-in" mints one unambiguous configuration where individual check-ins would each spawn one. The model rests on three relationship types — component hierarchies (is-part-of), version histories (is-derived-from, distinguishing *alternatives*/parallel branches from *derivatives*/same line), and **equivalences** (cross-representation identity between heterogeneous facets of the same real thing) — with structure kept separate from tool-owned content. And: versions are minted "at significant, semantically meaningful" points, "not every update."

② **How it maps.** This is the richest answer to "a grouping of functions, versioned, with its own history." A configuration is *the noun* — your code ingestor's version-manifest, a git commit, a curated grouping, and a synthesized local world are all the same kind of object: a first-class, identity-bearing binding of a group-version to member-versions. "Equivalences" is the kernel's cross-source correspondence (the same understanding as code, as trail, as discourse node). And "mint at meaningful points, not every update" is the commit-vs-save decision, stated in 1990.

③ **What it means.** Promote the version-manifest from bookkeeping to a **first-class configuration object** — because it is simultaneously the group-version thread *and* the exact noun the future Softland-native "commit" needs. Adopt static-vs-dynamic (pinned vs floating) as a deliberate per-binding choice, and adopt **group check-in** (one deliberate act mints one group version) as the antidote to propagation ambiguity — which in Rama also sidesteps the cross-partition atomicity problem, because the configuration is a *claim about a cut* (data), not a transactional freeze.

ORION — versioning a grouping-of-types, and the cascade it forces

Kim & Chou, VLDB (1988) — primary

① **What it worked through.** ORION versions schema and shows what happens when you version an interconnected group at member grain. It splits identity from value via a **generic instance** (shared OID) and **version instances**, with pinned-vs-floating explicit: "if the reference is to a version instance ... statically bound ... to a generic instance ... dynamically bound to a default." Facing "version the schema," they weighed three grains — whole lattice, per-class, dynamic views — and chose to version *the whole lattice as one object*, because per-class versioning **cascades**: "since each class object has SuperClasses and SubClasses attributes ... a new version must be created for each of the subclasses of C ... and the entire lattice." Group ↔ member is **effectivity-scoped via access scope**: "the access scope of a schema version SV is the set of objects created under SV and those ... inherited from ancestor schema versions" — membership is scope inheritance, not enumerated pins. Identity survives copy-on-write via a shared OID across an "anchor instance" and its copies.

② **How it maps.** The empirical proof of Molhado's "version proliferation" warning: naive member-grain versioning of a *connected* group degenerates into whole-group versioning, because the connections themselves are data that must change. This is the central risk in your "regroup functions freely, version the groupings" vision.

③ **What it means.** Don't version every grouping eagerly (total versioning) — it cascades. Use the **composition model** (selections over member versions) with explicit pinned/ floating/effectivity-scoped bindings, and let group versions be minted deliberately. ORION's *access scope* is also a third composition style worth remembering — membership by inherited scope, not by enumerated member pins — which may fit "a local world inherits most of its parent and overrides a few things" better than pinning.

Sciore — the unification, stated as a claim in 1994

VLDB Journal (1994) — primary

① **What it worked through.** Sciore argues versioning belongs *in the data model*, not in per-application code: "current database systems support versioning at a very low level ... this article demonstrates that application-independent versioning" can be a high-level data-model feature. The mechanism extends an OO data model so that "configurations can be specified conceptually and non-procedurally" — declarative

configuration selection rather than pinned enumeration — by treating "version sets ... multidimensionally," so a group-version resolves from coordinates over member version-sets. And the framing claim: it "integrates and generalizes ideas in CAD systems, CASE systems, and temporal databases."

② **How it maps.** This is the explicit statement that CAD assemblies, software configurations, and temporal databases are *one* theory — the general theory you asked whether existed. And "declarative, multidimensional configuration selection" is a more powerful version of your "situate and regroup functions in different ways": a grouping defined by *coordinates/queries* over member versions, not a hand-pinned list.

③ **What it means.** Validates that "versioning is its own layer" is a *data-model-level* claim, not an ingestor detail — which is exactly your current mental model. And declarative/ multidimensional configuration is the sophisticated end-state for Softland's regrouping vision: groupings as *queries over versioned members*, computed, not enumerated — which sits naturally on Rama PStates.

PLM effectivity — industry's forty-year answer to versioned assemblies

orey.github.io/papers/conf-mgt (2021) — *blog*

① **What it worked through.** Product Lifecycle Management has shipped "versioned grouping of versioned members" for decades. Membership is **serial-number-effectivity-scoped**: "brakes V1 are applicable to ... S/N in the range [1, N], brakes V2 ... [N+1, ∞)." An applicability link is *both* retrospective fact and prospective rule — "indicate that a specific product S/N is using a specific component [and] that, for future S/N ... the component to be used should be a certain version." It contrasts static "wired" assemblies ("when a version is wired with links, it will never change") against dynamic query-time filtering, and insists configuration be "managed at different levels ... variants and options at the component level ... applicability at the product level." Change is snapshot/copy-based with changes reified as links between versions.

② **How it maps.** A whole mature industry on exactly your problem, with a composition primitive the software literature underweights: **effectivity** — membership scoped by a range or condition, simultaneously a historical record and a forward rule.

③ **What it means.** Effectivity is a third binding mode beyond pinned/floating, and it's the right one for some Softland cases: "this grouping uses member-version V2 *from this point forward*" is a forward rule + an audit fact in one object. Worth carrying in the vocabulary alongside bound/generic.

Cross-cutting synthesis — what all of this means together

1. The two-layer model is the convergent answer, independently, everywhere. Immutable content-addressed *values* + a thin mutable *naming* layer on top: git refs, Irmin tags, Unison names, Dolt branches, Molhado nodes, Conradi-Westfechtel OID/VID. The kernel's container (name) / revision (value) split is not a bet — it's where everyone lands. What the kernel adds that most predecessors lacked: the value layer is *natively event-sourced* (Rama), so you keep history by default instead of bolting it on.

2. Versioning is two layers, not one — and you already half-have it. DSST's partial vs full persistence is the formal seam you felt. **Partial persistence** = Rama's arrival-time depot (a log; read any past, extend the tip) = *transaction time*. **Full persistence** = the minted, branchable, git-style version graph you want to add = *semantic time*, and = "infinite Softlands." The mint is an *act*; a **configuration** (Katz) is the object it produces.

So the layer to design is: deliberate mints producing first-class configuration objects, sitting above the always-on depot.

3. The composition question has three known answers and one named trap. For "versioned grouping of versioned members," choose the **composition model** (version members; a group is a selection over member versions) with per-binding mode — **pinned** (Katz static / ORION static / C&W bound), **floating** (dynamic / generic), or **effectivity-scoped** (PLM ranges, ORION access scope). *Avoid total versioning* (version every group eagerly): ORION and Molhado both document its failure mode — **version proliferation / cascade**, where one leaf change ripples new versions up the whole structure. And *avoid a single global version space* (Molhado product versioning): it makes "link to D at version vk" inexpressible, which would kill anchors and trails.

4. Four-level graph versioning provably needs ≥ 2 linked tracks. You cannot reconstruct group-level history from member-level history alone (Pelgrin's theorem; Wikidata's working braid of entity-chain-inside-global-chain). So the local-revision and global-revision tracks must both exist *from the start* — which argues for putting the configuration/group-version noun into the kernel **before** the graph ingestor forces it. Per-attribute opt-in versioning (Molhado) decides *which* tracks a thing carries.

5. Change representation is a projection, not an ontology. Snapshot (IC), delta (CB), and annotation (TB) are all derivable from one event-sourced substrate — Wikidata serves both diffs and full states over one store; HoTT patch theory makes "one theory, many models" a theorem. The kernel's "log is primary, projections are derived" is the mainstream-correct architecture. Two warnings ride along: **version-count is a real wall** (RDF systems die at 21k versions, time out at 1.3k) — vindicating commit-grain over save-grain — and **aggregated delta chains grow unbounded** unless you periodically re-materialize snapshots.

6. Declared beats inferred — and this is Softland's structural advantage. Every system divides on whether change *meaning* is declared (MolhadoRef's recorded refactorings, Unison's metadata rename) or inferred (git's similarity heuristic). The kernel already rules: inference is never accepted as truth at ingest. The asymmetry no prior system had: **the agent trails are a declared-operation log** — edits *with narrated intent* — so rename/refactor continuity, the literature's hardest open problem, has a producer Softland uniquely owns. MolhadoRef needed to own an IDE; Softland owns the agents.

7. Preserved disagreement can be a data structure. Pijul represents conflicts as first-class values (the unconstructed pushout). That is the formal backing for Softland's late-bound consensus / plurality-as-fourth-axis: a disagreement is a representable kernel state, not an error to resolve away or silently overwrite.

What this changes for the code ingestor (concrete, against the slice-1 contract)

The research **validates more than it overturns**. Stable container + revisions = OID/VID verbatim. Commit-grain minting = Katz's "meaningful points, not every update," and the version-count wall actively vindicates it. Refusing git rename detection is right *because* the correct answer is declaration, not better inference. Whole-file artifacts now, forms later = MolhadoRef's own granularity-floor retreat. The earned amendments:

1. **Dedup artifacts by content, link occurrences** (Irmin/Prolly/Unison) — one value per distinct bytes; same bytes at a new path = same value, new place $\Rightarrow$ half of rename continuity as *fact*, not inference.
2. **Promote the version-manifest to a first-class configuration object** (Katz) — it is the group-version thread *and* the noun the future Softland-native commit needs; unify them now, at zero cost.

3. **Make the two times explicit** (XTDB) — commit author-time (valid) vs ingest-time (transaction); the exactness flag becomes a projection of this, not a special case.
4. **Name-threads (codeq def-names) as slice-2 claims**, with trail-extracted continuity declarations (MolhadoRef-style) as their upgrade path.
5. **For the regrouping vision: composition model only** (pinned / floating / effectivity- scoped selections), never total versioning, never one global space.

And the one reordering this whole chat earned: the versioning **vocabulary** — item/OID/VID, configuration, mint, the two times, bound/generic/effectivity, partial vs full persistence — belongs in a kernel-physics doc **above** the ingestor contracts, because the code ingestor is merely the first place the question got loud; Roam, groupings, and graphs all route through the same nouns, and Pelgrin proves the group-track can't be safely retrofitted.

Open problems the literature itself flags as unsolved

- **Semantic merge** — merging declared refactorings with ordinary edits (MolhadoRef 2006 future work; HoTT merge laws unproven; Darcs conflict story deferred). Don't promise it.
- **History-length-independent performance** — delta chains and aggregated changesets degrade; version-count walls are real (21k versions = failure).
- **Cross-version references inside a global version space** (Molhado) — directly threatens anchors/trails; solved here by *refusing* the global space.
- **Change propagation scope & DAG ambiguity** for composites (Katz) — resolved only operationally (group check-in), never automatically.
- **No standard query model for temporal graphs** (RDF surveys) — OSTRICH's VM/DM/VQ is the best candidate vocabulary; adopt it.

People index (the non-famous seminal ones)

Jacky Estublier (Adele, 1985) · Klaus Dittrich et al. (DAMOKLES, 1986) · Randy Katz (the 1990 configuration survey) · Won Kim & Hong-Tai Chou (ORION, 1988) · Edward Sciore (the 1994 unification) · Tien N. Nguyen (Molhado, MolhadoRef) · Neil Sarnak (PhD thesis behind DSST persistence) · Ira Goldstein & Daniel Bobrow (PIE, Xerox PARC 1981 — layers/contexts, the Kay-lineage ancestor of dynamic configurations) · Samuel Mimram & Cinzia Di Giusto (categorical patch theory) · Carlo Angiuli, Ed Morehouse, Dan Licata, Robert Harper (HoTT patch theory) · Pierre-Étienne Meunier & Florent Becker (Pijul) · Thomas Pellissier-Tanon & Fabian Suchanek (Wikidata history) · Olivier Pelgrin, Luis Galárraga, Katja Hose (RDF archiving) · Reidar Conradi & Bernhard Westfechtel (the survey spine).

Sources by quality

Primary: DSST persistence (JCSS 1989) · Conradi & Westfechtel (CSUR 1998) · Katz (CSUR 1990) · ORION (VLDB 1988) · Sciore (VLDBJ 1994) · Molhado (SCM-12 2005) · MolhadoRef (ASE 2006) · Monticello (2013) · Unison docs · Pijul docs · Mimram & Di Giusto (arXiv 1311.3903) · Angiuli et al. HoTT patch theory · Dolt Proly docs · TerminusDB docs · XTDB bitemporality docs · OSTRICH (JWS 2018) · RDF archiving (SWJ 2019) · Pelgrin et al. (SWJ 2021) · Wikidata history (ESWC 2019). **Secondary / blog:** Irmin via The Morning Paper · Darcs patch theory (Wikibooks) · DoltHub blogs (2020, 2021) · PLM effectivity

(orey.github.io 2021). **Fetchd but empty:** ResearchGate "fine-grained flexible version control" (2008) — no extractable content.